Logistic Regression

November 4, 2026

0.1 2 Logistic Regression

In this section we look at three different logistic regression models for classifying myocardial infarctions patients in terms of mortality as a means of assessing medical risk. We look at preprocessed regression, regularized regression, and splines regression, considered different performance metrics on each model at different thresholds, and considered the best model to use given the medical context.

0.2 2.1 Preprocessing

[2]:
```
import pandas as pd

train = pd.read_csv("training_data.csv")
validation = pd.read_csv("validation_data.csv")
test = pd.read_csv("test_data.csv")

target = "mortality"

# Training data
x_train = train.drop(columns=[target])
y_train = train[target]

# Validation data
x_val = validation.drop(columns=[target])
y_val = validation[target]

# Test data
x_test = test.drop(columns=[target])
y_test = test[target]




#removing constant columns
from sklearn.feature_selection import VarianceThreshold

remove_constant = VarianceThreshold().set_output(transform="pandas")
```

```
x_train = remove_constant.fit_transform(x_train)
x_val = remove_constant.transform(x_val)
x_test = remove_constant.transform(x_test)
```

Imputation Our data had a lot of missingness, so we used simple imputation to make it useable. Since missing values are replaced by the median in the data, we do not expect values that were previously missing to change the predicted outcome by much.

[3]:
```
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

continuous = [
    "AGE",
    "S_AD_KBRIG", "D_AD_KBRIG",
    "S_AD_ORIT", "D_AD_ORIT",
    "K_BLOOD", "NA_BLOOD",
    "ALT_BLOOD", "AST_BLOOD", "KFK_BLOOD",
    "L_BLOOD", "ROE"
]

continuous = [c for c in continuous if c in x_train.columns]
categorical = [c for c in x_train.columns if c not in continuous]

imputer = ColumnTransformer([
    ("continuous", SimpleImputer(strategy="median"), continuous),
    ("categorical", SimpleImputer(strategy="most_frequent"), categorical)
]).set_output(transform="pandas")

x_train_imp = imputer.fit_transform(x_train)
x_val_imp = imputer.transform(x_val)
x_test_imp = imputer.transform(x_test)
```

The following section encodes the 5 types of chronic heart failure the patient can experience: 0-no heart failure, 1,2,3,4-different types of heart failure. Although 0 is less severe than 1, 2 is not less severe than 4, and so we need to treat this data as categorical, and so we one-hot encode it, separating it into 5 binary features rather than 1.

[4]:
```
#onehot encoding nominal features
x_train_enc = pd.get_dummies(
    x_train_imp, columns=["categorical__ZSN_A"], dtype=int
)

x_val_enc = pd.get_dummies(
    x_val_imp, columns=["categorical__ZSN_A"], dtype=int
)
x_val_enc = x_val_enc.reindex(columns=x_train_enc.columns, fill_value=0)
```

```
x_test_enc = pd.get_dummies(
    x_test_imp, columns=["categorical__ZSN_A"], dtype=int
)
x_test_enc = x_test_enc.reindex(columns=x_train_enc.columns, fill_value=0)
```

0.3 2.2 Feature Filtering

In this section we look at the most basic way to implement logistic regression as a baseline for the other 2 models. We use the f-statistic to identify the k most important features in the data on which to fit a model. The f-statistic measures which features are most concentrated in their class, which is a good indication of how informative each feature will be when predicting class. However, this method is unable to identify how useful features may be as a combination, and so may discard useful information. In the following code we find the best 5,10,20,30 and 50 features.

[5]:
```
from sklearn.feature_selection import SelectKBest, f_classif

selected = {}

for k in [5, 10, 20, 30, 50]:
    selector = SelectKBest(f_classif, k=k).set_output(transform="pandas")
    selected[k] = {
        "train": selector.fit_transform(x_train_enc, y_train),
        "val": selector.transform(x_val_enc)
    }
```

We now train a logistic regression model on each subset of features and compare the performance using training and validation loss.

[6]:
```
from sklearn.base import clone
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss

clf1 = make_pipeline(
    StandardScaler(),
    LogisticRegression(
        C=np.inf,
        solver="lbfgs",
        max_iter=5000
    ),
)

results1={}
models1={}
for k in selected:
    model1 = clone(clf1)
```

```
    model1.fit(selected[k]["train"], y_train)
    p_fit = model1.predict_proba(selected[k]["train"])
    p_val = model1.predict_proba(selected[k]["val"])
    models1[k]=model1
    results1[k] = {
        "training_log_loss": log_loss(
            y_train, p_fit, labels=model1.classes_
        ),
        "validation_log_loss": log_loss(
            y_val, p_val, labels=model1.classes_
        )
    }
pd.DataFrame(results1)
```

[6]:

	5	10	20	30	50
training_log_loss	0.364860	0.324464	0.283549	0.256291	0.236690
validation_log_loss	0.404114	0.365087	0.329454	0.347675	0.360251

Based on this, 20 seems to be the most appropraite number of features, so we proceed with the 20 best features. We train the model on all the data before prediction.

[7]:
```
#combining validation and training data
x_full = pd.concat([x_train_enc, x_val_enc], ignore_index=True)
y_full = pd.concat([y_train, y_val], ignore_index=True)
```

[8]:
```
cols = selected[20]["train"].columns

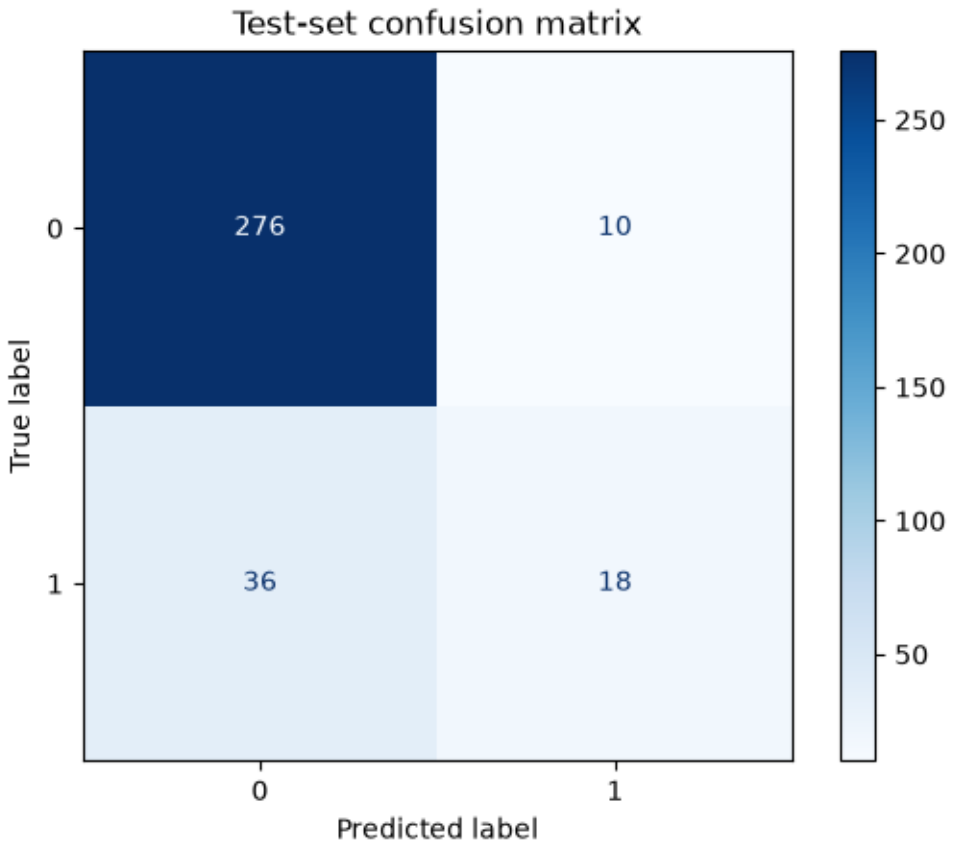
x_full_20 = x_full.loc[:, cols]
x_test_20 = x_test_enc.loc[:, cols]

final_model1 = clone(models1[20])
final_model1.fit(x_full_20, y_full)

print(f"Test accuracy: {final_model1.score(x_test_20, y_test):.2%}")
```

```
Test accuracy: 86.47%
```

[9]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_estimator(
    final_model1,
    x_test_20,
    y_test,
    cmap="Blues",
    values_format="d"
)
plt.title("Test-set confusion matrix")
plt.show()
```



Although our test is 86% accurate, it is missing a lot of deaths, meaning it is potentially seriously underestimating medical risk. As our data is unbalanced, with far more surviving patients, a naive model will usually underpredict death, as this one does, even while achieving good overall accuracy. We can see the data is only 33% accurate on patients who died, although predicting death is the entire aim of the model, which suggests this model is very poor for our purposes. We will look later at changing the threshold for this model to achieve better accuracy on positive cases, and compare it to other logistic regression models on metrics which penalize false negatives.

0.4 2.4 Regularized Regression

In this section we use L1 regularized regression with different penalty parameters to fit a model for predicting mortality in the patients. Regularization punishes model size, which in this case means size of coefficients, so it effectively uses a smaller set of the features, as in 1.2, but uses a penalty term in the loss function instead of filtering to identify the most informative features.

This code implements regularized regression with a variety of penalty coefficients, which we will choose from later based on the validation data.

[10]:
```
cs = np.logspace(-3, 2, 10)

clf = make_pipeline(
```

```
    StandardScaler(),
    LogisticRegression(
        l1_ratio=1,
        solver="liblinear",
        tol=1e-4,
        max_iter=5000,
        fit_intercept=True,
        random_state=42
    )
)

models2 = {}
results = []

for c in range(len(cs)):
    model = clone(clf)
    model.set_params(logisticregression__C=cs[c])
    model.fit(x_train_enc, y_train)

    p_fit = model.predict_proba(x_train_enc)
    p_val = model.predict_proba(x_val_enc)

    models2[c] = model

    results.append({
        "C": cs[c],
        "training_log_loss": log_loss(
            y_train, p_fit, labels=model.classes_
        ),
        "validation_log_loss": log_loss(
            y_val, p_val, labels=model.classes_
        )
    })

results = pd.DataFrame(results)
results
```

[10]:

	C	training_log_loss	validation_log_loss
0	0.001000	0.693147	0.693147
1	0.003594	0.610042	0.609776
2	0.012915	0.428080	0.436562
3	0.046416	0.295266	0.345645
4	0.166810	0.223693	0.327234
5	0.599484	0.199170	0.360883
6	2.154435	0.194179	0.389173
7	7.742637	0.192797	0.406245
8	27.825594	0.192466	0.415783

```
9   100.000000            0.192395              0.423596
```

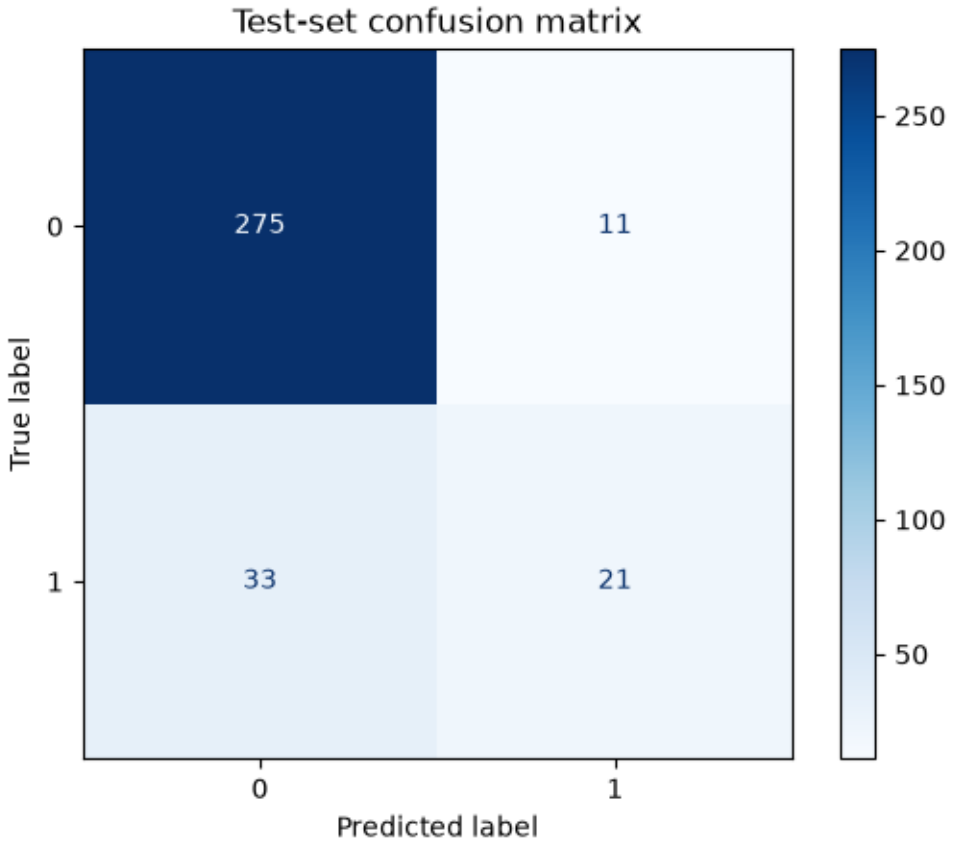
Based on this we select model 4 with a C value of 0.166810, and evaulate it's accuracy on the test data.

[11]:
```
final_model = clone(models2[4])
final_model.fit(x_full, y_full)
print(f"Test accuracy: {final_model.score(x_test_enc, y_test):.2%}")
```

```
Test accuracy: 87.06%
```

We can see that accuracy has improved marginally, but we don't yet know if this has improved our accuracy on positive cases, so we plot a confusion matrix.

[12]:
```
ConfusionMatrixDisplay.from_estimator(
    final_model,
    x_test_enc,
    y_test,
    cmap="Blues",
    values_format="d"
)
plt.title("Test-set confusion matrix")
plt.show()
```



Based on this we can see that we are correctly identifying a higher portion of positive cases (39%), which is a big improvement on the previous model, especially considering that the overall accuracy has only improved marginally. Given that we have largely improved our recall without sacrificing specificity, we can be fairly confident that this is a better model. However, 39% is still not a good rate for identifying mortality, so we need to improve upon the model, and adapt it to the context.

0.5 2.5 Spline Regression

This section looks at non-linear logistic regression: specifically cubic B-spline regression where instead of estimating a set of coefficients for each feature vector, we estimate a set of coefficients for polynomial functions on the feature vectors. Each polynomial is fitted several times over different intervals inside each feature: for exmaple you might fit one polynomial over the age range 0-20, one over the range 20-40, and one over the range 40-100. The number of intervals (or number of knots) is a hyperparameter we estimate by minimising validation loss. Most of the features were categorical, so these were modelled linearly instead. We have also added L2 regularisation: L2 performs better on related predictors, and we are now treating different intervals of one feature as separate predictors, meaning they are strongly related, and so it was a better choice for splines regression. The L2 penalty parameter, c, is also cycled through and is chosen based on minimising validation loss.

[13]:
```
from sklearn.preprocessing import SplineTransformer

spline_cols = [
    f"continuous__{c}"
    for c in continuous
    if f"continuous__{c}" in x_train_enc.columns
]


ns = [3, 4, 5]
cs = np.logspace(-3, 2, 10)

clf = make_pipeline(
    ColumnTransformer(
        transformers=[
            (
                "splines",
                SplineTransformer(
                    degree=3,
                    knots="quantile",
                    include_bias=False,
                    extrapolation="linear",
                ),
                spline_cols,
            ),
```

```
        ],
        remainder="passthrough",
    ),
    StandardScaler(),
    LogisticRegression(
        solver="lbfgs",
        fit_intercept=True,
        tol=1e-4,
        max_iter=3000,
    ),
)

models = {}
results = []

for n in ns:
    for c in cs:

        model = clone(clf)
        model.set_params(
            columntransformer__splines__n_knots=n,
            logisticregression__C=c,
        )

        model.fit(x_train_enc, y_train)

        p_train = model.predict_proba(x_train_enc)
        p_val = model.predict_proba(x_val_enc)

        models[(n, c)] = model

        results.append({
            "n_knots": n,
            "C": c,
            "training_log_loss": log_loss(
                y_train, p_train, labels=model.classes_
            ),
            "validation_log_loss": log_loss(
                y_val, p_val, labels=model.classes_
            ),
        })

results = pd.DataFrame(results)

# Choose the fitted model with minimum validation loss.
best_row = results.loc[results["validation_log_loss"].idxmin()]
best_key = (int(best_row["n_knots"]), float(best_row["C"]))
```

```
best_model = models[best_key]

display(results.sort_values("validation_log_loss").head())
print(
    f"Best: n_knots={best_key[0]}, C={best_key[1]:.4g}, "
    f"validation log loss={best_row['validation_log_loss']:.4f}"
)
```

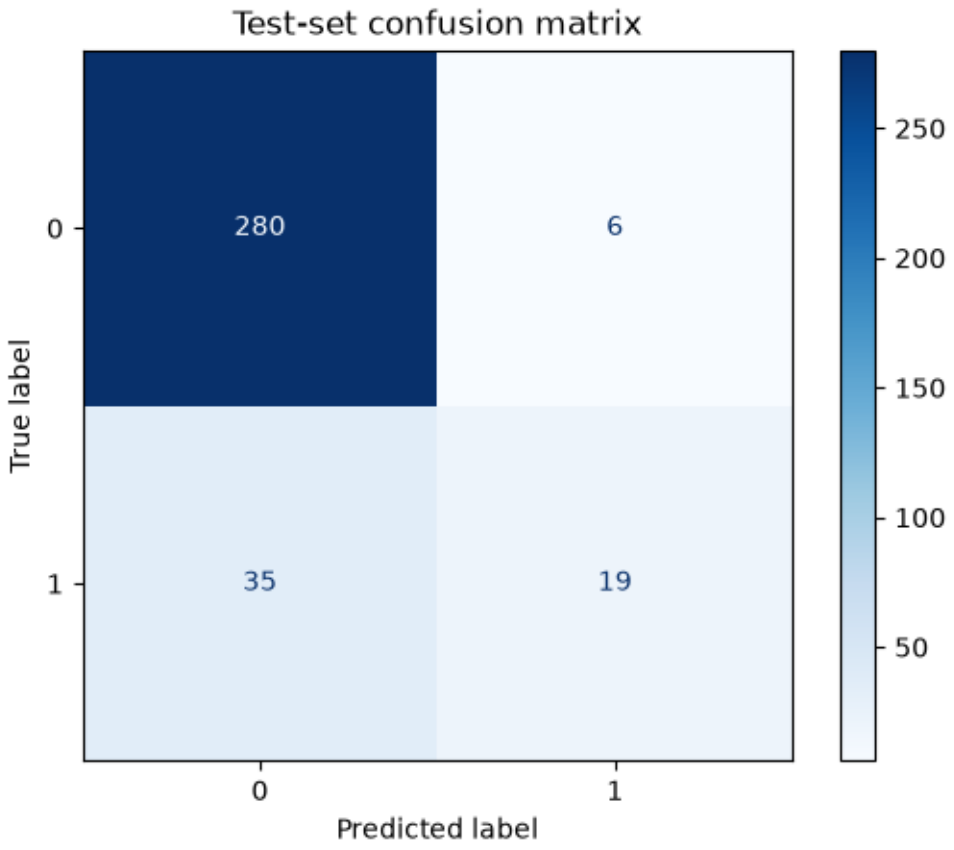
	n_knots	C	training_log_loss	validation_log_loss
22	5	0.012915	0.223524	0.310955
12	4	0.012915	0.228469	0.315146
2	3	0.012915	0.230211	0.315213
21	5	0.003594	0.270523	0.318238
11	4	0.003594	0.274356	0.323833

```
Best: n_knots=5, C=0.01292, validation log loss=0.3110
```

[14]:
```
final_model = clone(best_model)
final_model.fit(x_full, y_full)
print(f"Test accuracy: {final_model.score(x_test_enc, y_test):.2%}")
```

```
Test accuracy: 87.94%
```

[15]:
```
ConfusionMatrixDisplay.from_estimator(
    final_model,
    x_test_enc,
    y_test,
    cmap="Blues",
    values_format="d"
)
plt.title("Test-set confusion matrix")
plt.show()
```



While this model was more accurate, it did not have a higher accuracy on the positive cases (sensitivity of 35%), but instead hada higher specificity, and so it did not improve on the previous model for our chosen application. In the next section we compare each logistic regression model on a variety of different metrics which gives us a more complete comparison than sensitivity and accuracy alone.

0.6 2.6 Performance Metrics

This section looks at comparing the models in this section on different performance metrics, and at changing the thresholds of each model to achieve a higher sensitivity.

Below are the ROC curves for each model: these plot the true positive rate over the false positive rate, assessing the model over all thresholds. We can see that the models perform similarly: with spline regression having the best trade off at the high true positive/ false negative side of the graph. This contexualises our results from earlier: although spline regression is outperformed by L1 regularized regression at the default threshold of 0.5, it outperforms it at a lower threshold.

[16]:
```
from sklearn.metrics import roc_curve, roc_auc_score
```

```
models_to_plot = [
    ("Filtered model", final_model1),
    ("Regularised regression", final_model),
    ("Spline model", best_model),
]

fig, ax = plt.subplots(figsize=(8, 6))
auc_results = []

y_true = np.asarray(y_test).ravel() == 1

for name, model in models_to_plot:
    X = x_test_enc.loc[:, model.feature_names_in_]

    death_index = np.flatnonzero(model.classes_ == 1).item()
    p_death = model.predict_proba(X)[:, death_index]

    fpr, tpr, _ = roc_curve(y_true, p_death)
    auc_value = roc_auc_score(y_true, p_death)

    ax.plot(
        fpr,
        tpr,
        linewidth=2,
        label=f"{name} (AUC = {auc_value:.4f})",
    )

    auc_results.append({
        "Model": name,
        "ROC_AUC": auc_value,
    })

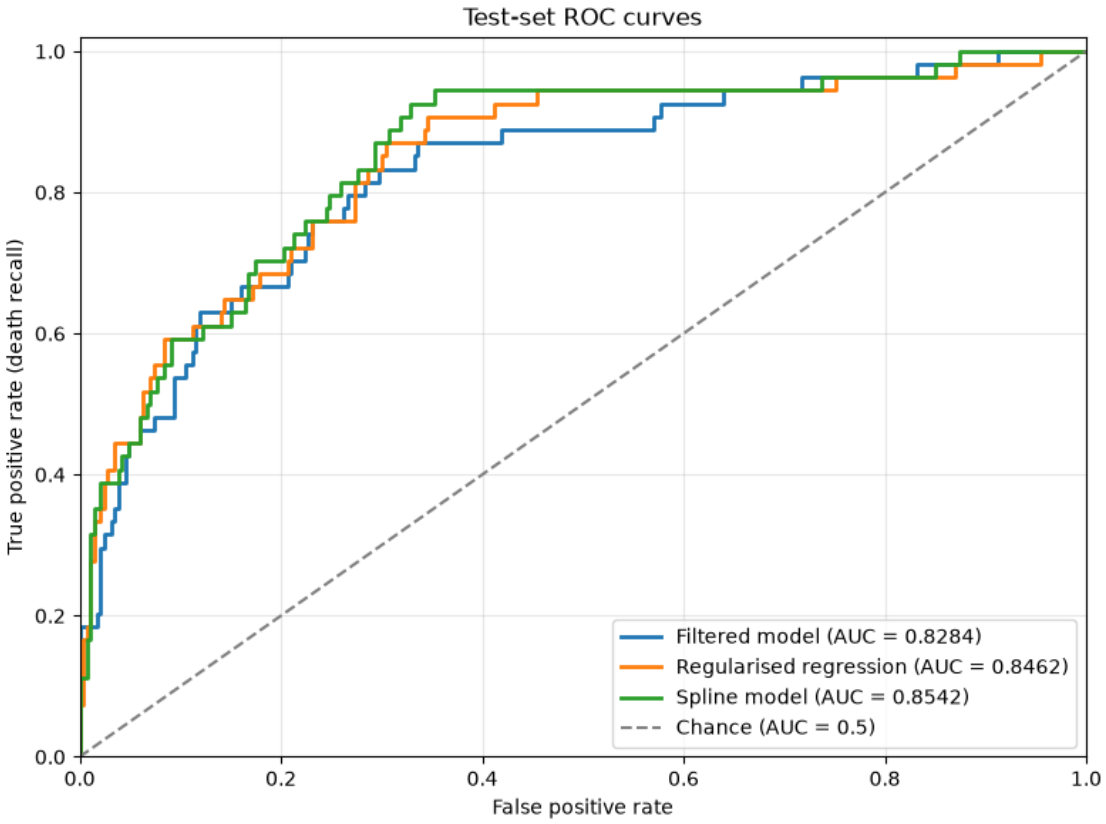
ax.plot(
    [0, 1], [0, 1],
    linestyle="--",
    color="grey",
    label="Chance (AUC = 0.5)",
)

ax.set(
    xlabel="False positive rate",
    ylabel="True positive rate (death recall)",
    title="Test-set ROC curves",
    xlim=(0, 1),
    ylim=(0, 1.02),
)
```

```
ax.legend(loc="lower right")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

auc_table = (
    pd.DataFrame(auc_results)
    .sort_values("ROC_AUC", ascending=False)
    .reset_index(drop=True)
)

display(auc_table.round(4))
```



	Model	ROC_AUC
0	Spline model	0.8542
1	Regularised regression	0.8462
2	Filtered model	0.8284

The table displays the area under curve, which measures the probability that a randomly selected positive datapoint from our test data receives a higher score (probability for us) than a randomly selected negative, which gives us a good measure of how well we are assigning probabilities before classifying.

We can dervive this as follows:

We know that true positive rate $TPR = 1 - P(S^+ < t) = 1 - F_+(t)$ where S^+ is the score given to a positive value, F_+ is the CDF of the score, and t is the threshold. Similarly, false positive rate, $FPR = 1 - P(S^- < t) = 1 - F_+(t)$, since these measure the probability of being correctly anf incorrectly classified for a given threshold. We then have that

$$AUC = \int_{\text{inf}}^{-\text{inf}} TPR \frac{dFPR}{dt} dt = \int_{\text{inf}}^{-\text{inf}} 1 - F_+(t)(-f(t))dt = \int_{-\text{inf}}^{\text{inf}} 1 - F_+(t)(f(t))dt$$

$$= \int_{-\text{inf}}^{\text{inf}} P(S^+ > t)(f(t))dt = \int_{-\text{inf}}^{\text{inf}} P(S^+ > S^- | S^- = t)(f(t))dt = P(S^+ > S^-).$$

Proof is summarised from [3].

Based on this it appears that the spline classifier does the best, followed by the regularized classifier, and then the filtered classifier. We can see that although the filtered classifier only performs slightly worse than the regularized and spline classifier in terms of accuracy, the difference in AUC is quite high, suggesting that much of the information that we lost when selecting the most informative features was useful for the positive cases, but this was overlooked in the validation loss because our data is unbalanced.

This AUC graph provides optimism for threshold changing, because we can see the graph continues to be far from the chance line across all thresholds and plateaus at a lower threshold (higher false positive rate), suggesting that at a certain low threshold we our false posotive rate will increase without increasing our true positive rate, making thresholds below this point useless. In our testing we will try to find the lowest threshold without a steep drop off in precision. This will probably not come from the filtered model which plateaus earlier. We now look at different thresholds for each model, and compare the new threshold models on 7 different performance metrics.

[17]:
```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    matthews_corrcoef,
    balanced_accuracy_score,
    roc_auc_score,
)


metrics = [
    "Accuracy", "Precision", "Recall", "F1",
    "Matthews CC", "Balanced accuracy"
]

models_to_check = [
    ("Filtered model", models1[20]),
    ("Regularized model", models2[4]),
```

```
    ("Spline model", best_model),
]

thresholds = [0.5, 0.4, 0.3, 0.2, 0.1]
rows = []

# Use validation data to investigate thresholds-not the test set
y_true = np.asarray(y_val).ravel() == 1

for name, model in models_to_check:
    X = x_val_enc.loc[:, model.feature_names_in_]

    death_index = np.flatnonzero(model.classes_ == 1).item()
    p_death = model.predict_proba(X)[:, death_index]

    for threshold in thresholds:
        predictions = p_death >= threshold

        tn, fp, fn, tp = confusion_matrix(
            y_true, predictions, labels=[False, True]
        ).ravel()

        rows.append({
            "Model": name,
            "Threshold": threshold,
            "Accuracy": accuracy_score(y_true, predictions),
            "Precision": precision_score(
            y_true, predictions, zero_division=0
            ),
            "Recall": recall_score(
            y_true, predictions, zero_division=0
            ),
            "F1": f1_score(
            y_true, predictions, zero_division=0
            ),
            "Matthews CC": matthews_corrcoef(y_true, predictions),
            "Balanced accuracy": balanced_accuracy_score(
            y_true, predictions
            )
            })

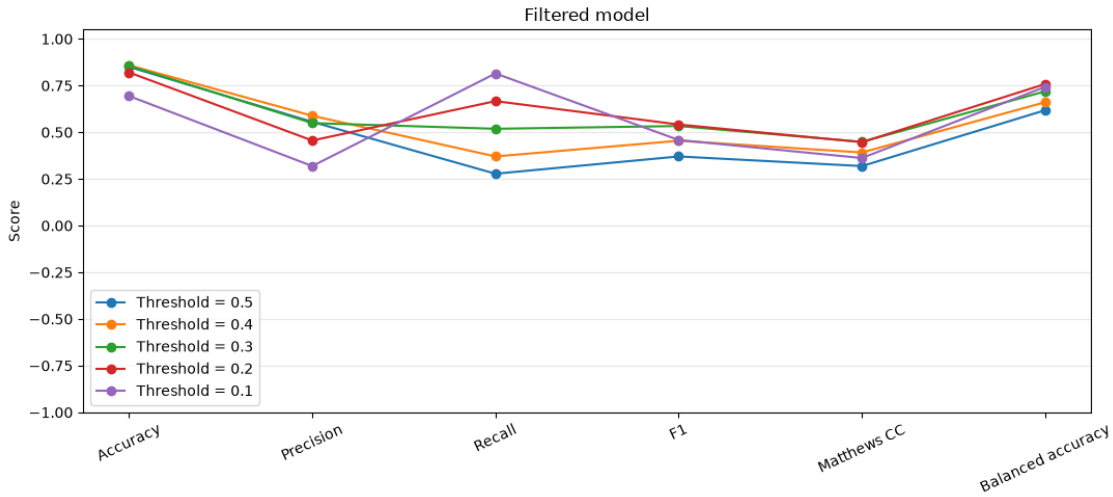

    model_results = pd.DataFrame(rows)
    model_results = model_results[model_results["Model"] == name]

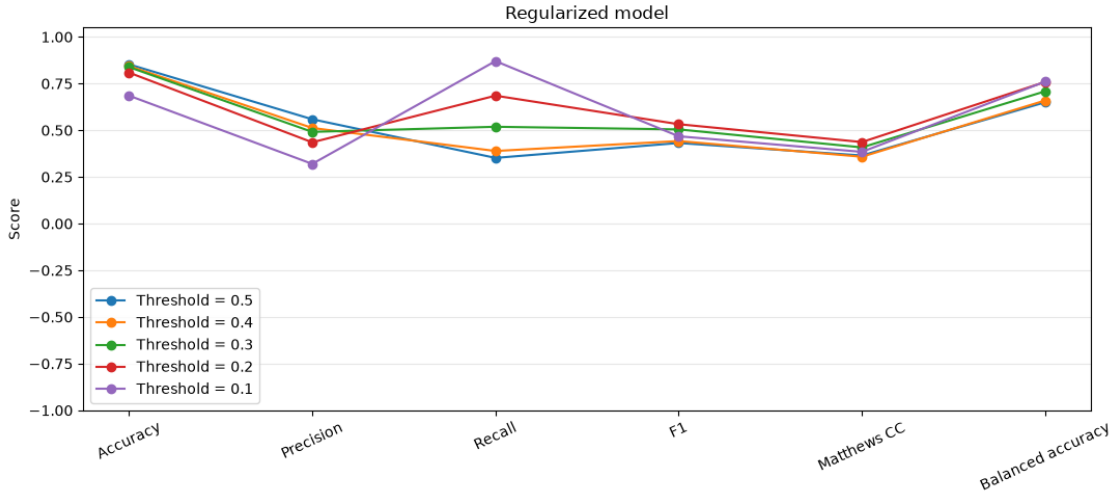
    fig, ax = plt.subplots(figsize=(11, 5))
```

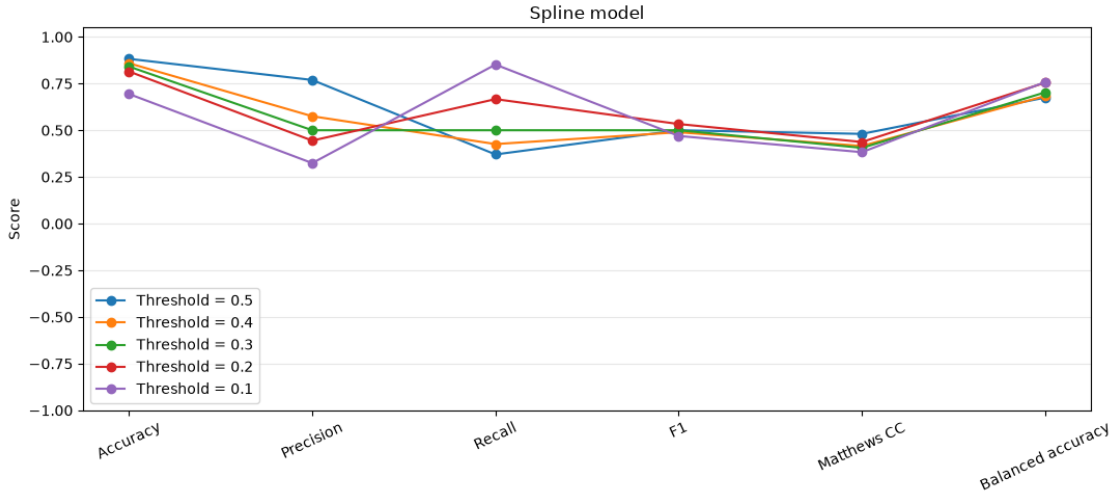
```
    for _, row in model_results.iterrows():
        ax.plot(
            metrics,
            row[metrics].to_numpy(dtype=float),
            marker="o",
            label=f"Threshold = {row['Threshold']:.1f}",
        )

    ax.set_title(name)
    ax.set_ylabel("Score")
    ax.set_ylim(-1, 1.05)
    ax.tick_params(axis="x", rotation=25)
    ax.legend()
    ax.grid(axis="y", alpha=0.3)
    fig.tight_layout()
    plt.show()

#recall_results = pd.DataFrame(rows)
#display(recall_results.round(3))
```







We can see that threshold changing has had a dramatic impact on sensitivity (or recall), but naturally this is coming at the cost of precision. We can get more insight from the balanced accuracy, f1 and Matthew's CC scores, which all measure accuracy by considering both the true positive and true negative rates (as discussed in Section 1). We can see that the lower threshold models are outperforming the highest threshold models on everything except precision and accuracy, but these measures can be obscured by the number of true positives since the data is unbalanced. When the threshold is changed, matthews cc f1 and balanced accuracy remain very similar for the spline and regularized models, even though recall and precision change, but are more variable for the simpler model, rel Overall we can see that using a lower threshold is justified because we gain a lot of recall without losing a lot in the other metrics. However, the spline moel loses more in precision when the threshold is adjusted than the other 2 models, so the best model to use may be a regularized model with threshold 0.1, or a spline model with threshold 0.2. We have chosen the regularized model with threshold 0.1 to compare to the models from other sections, but first we

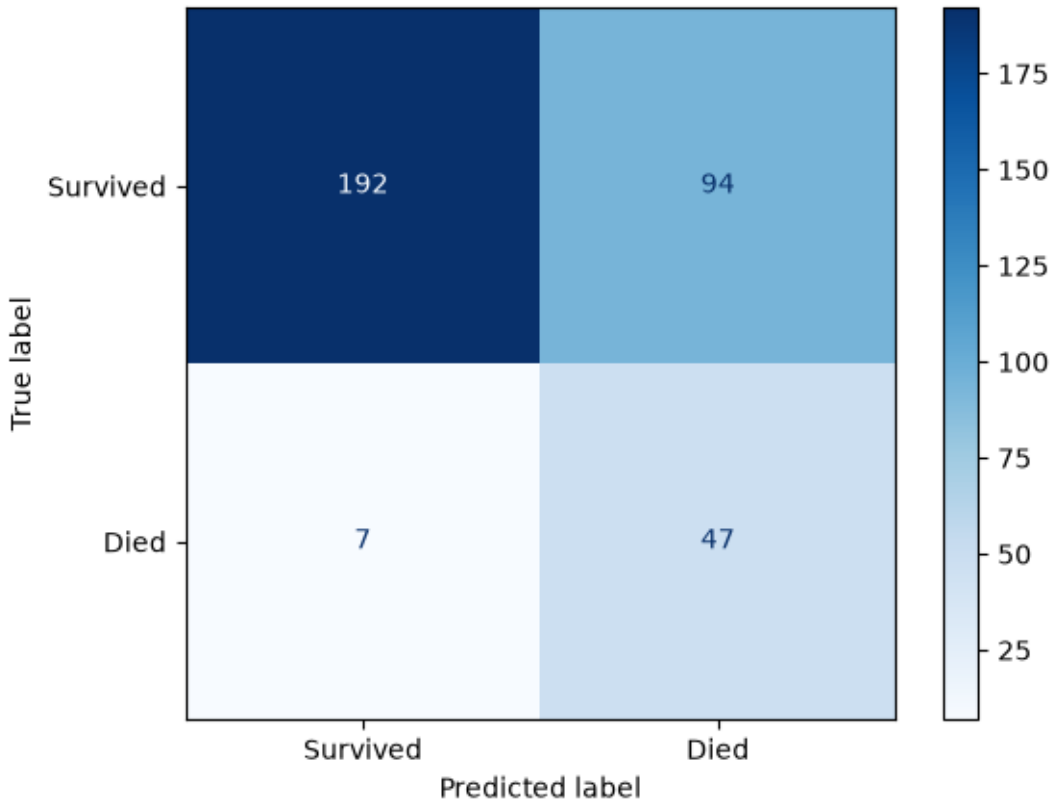
need to test it on the test data.

[18]:
```
from sklearn.frozen import FrozenEstimator
from sklearn.model_selection import FixedThresholdClassifier

model_01 = FixedThresholdClassifier(
    estimator=FrozenEstimator(final_model),
    threshold=0.1,
    response_method="predict_proba",
    pos_label=1,
)
```

[19]:
```
ConfusionMatrixDisplay.from_predictions(
    y_test,
    model_01.predict(x_test_enc),
    labels=[0, 1],
    display_labels=["Survived", "Died"],
    cmap="Blues",
    values_format="d",
)

plt.show()
```



[22]:
```
y_pred = model_01.predict(x_test_enc)
y_prob = model_01.predict_proba(x_test_enc)[:, list(model_01.classes_).index(1)]

results = pd.DataFrame([{
    "Accuracy": accuracy_score(y_test, y_pred),
    "Precision": precision_score(y_test, y_pred, zero_division=0),
    "Recall": recall_score(y_test, y_pred, zero_division=0),
    "F1": f1_score(y_test, y_pred, zero_division=0),
    "MCC": matthews_corrcoef(y_test, y_pred),
    "Balanced Accuracy": balanced_accuracy_score(y_test, y_pred),
    "ROC-AUC": roc_auc_score(y_test, y_prob)
}]).round(4)

results.to_csv("model_01_metrics.csv", index=False)

results
```

[22]:

	Accuracy	Precision	Recall	F1	MCC	Balanced Accuracy	ROC-AUC
0	0.7029	0.3333	0.8704	0.4821	0.4019	0.7708	0.8462

Finally, I have fitted the model lower threshold regularized model on the test data and achieved a model with good recall, and relatively good accuracy.

1. Golovenkin, S., Shulman, V., Rossiev, D., Shesternya, P., Nikulina, S., Orlova, Y., & Voino-Yasenetsky, V. (2020). Myocardial infarction complications [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C53P5M
2. B-Splines, https://en.wikipedia.org/wiki/B-spline
3. Estimating weighted areas under the ROC curve (2020), https://proceedings.neurips.cc/paper/2020/file/5781a2637b476d781eb3134581b32044-Paper.pdf
4. https://blog.minitab.com/en/blog/adventures-in-statistics-2/understanding-analysis-of-variance-anova-and-the-f-test
5. https://www.datacamp.com/tutorial/understanding-logistic-regression-python?utm_cid=23340058065&utm_aid=192632748929&utm_campaign=230119_1-ps-dscia$_{\text{dsa-tofu}}$python_2-b2c_3-europe_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&utm_loc=9045630-&utm_mtd=-c&utm_kw=&utm_source=google&utm_medium=paid_search&utm_content=ps-dscia$_{\text{europe-en}}$dsa$_{\text{tofu}}$tutorial~python&gad_source=1&gad_campaignid=23340058065&gbraid=0AAAADQ9
6. https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression